Set Interface

- `Set` is a **Collection** that contains **no duplicate elements**.
- It **extends the Collection interface**, so it has all the Collection methods.
- **Key difference from List/ArrayList:**
  - No duplicates allowed
  - No index-based access
  - Ordering depends on implementation

Common Set Implementations

Class	Order	Notes
`HashSet`	Unordered	Fast, allows one `null`
`LinkedHashSet`	Insertion order	Preserves order of insertion
`TreeSet`	Sorted	Natural ordering or via Comparator, **no null allowed**

Key Methods (Inherited from Collection)

- Set inherits all Collection methods
- Does NOT add new methods
- it changes the behavior from collection

Method	Behavior in Set
`add(E e)`	Adds element **only if not already present**
`remove(Object o)`	Removes element
`contains(Object o)`	Checks if element exists
`iterator()`	Returns iterator, order depends on Set type
`addAll(Collection<? extends E> c)`	Adds **only unique** elements from another collection
`clear()`	Removes all elements
`size()`	Number of elements

Example: Set vs List

```
import java.util.*;

public class SetExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Apple"); // duplicate allowed

        Set<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Apple"); // duplicate ignored

        System.out.println("List: " + list); // [Apple, Apple]
        System.out.println("Set: " + set);   // [Apple]
    }
}
```

Queue Interface

A **Queue** is a collection designed for **holding elements before processing**.
It follows the **FIFO (First In, First Out)** principle --- the first element added is the first removed.

Real-life Analogy

Think of a line at a ticket counter --- the person who comes first gets served first.

Core Queue Methods

Method	Description	Behavior When Empty / Full
`offer(E e)`	Adds element to the queue	Returns `false` if it fails
`add(E e)`	Adds element to the queue	Throws exception if it fails
`poll()`	Removes and returns head	Returns `null` if empty
`remove()`	Removes and returns head	Throws `NoSuchElementException`
`peek()`	Retrieves head without removing	Returns `null` if empty
`element()`	Retrieves head without removing	Throws `NoSuchElementException`

Tip:
Use `offer`, `poll`, and `peek` for **safe, non-exception** operations.

Example --- Using Queue with LinkedList

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        queue.offer("A");
        queue.offer("B");
        queue.offer("C");

        System.out.println("Queue: " + queue);
        System.out.println("Head: " + queue.peek());  // A

        System.out.println("Removed: " + queue.poll()); // removes A
        System.out.println("After removal: " + queue);  // [B, C]
    }
}
```

Output:

```
Queue: [A, B, C]
Head: A
Removed: A
After removal: [B, C]
```

Queue vs List

Feature	`List`	`Queue`
Access Type	Indexed	FIFO
Add Element	`add(index, e)`	`offer(e)`
Remove Element	`remove(index)`	`poll()`
Peek Element	`get(index)`	`peek()`
Typical Use	Store and access by index	Process elements in order

Interface	Purpose
`Queue<E>`	Represents a **first-in-first-out (FIFO) queue**. Elements are **added at the end** and **removed from the front**.
`Deque<E>`	Represents a **double-ended queue**. Elements can be **added or removed from both ends** (front and back).

Collections.sort()

It's a static method in the `java.util.Collections` utility class.

- Purpose: sort elements of a List in-place.

1. Sort using natural order
   Collections.sort(List list)

Example:

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 1, 3));
Collections.sort(numbers); // Ascending order
System.out.println(numbers); // [1, 3, 5]
```

2. Sort using a custom comparator

Example (descending order):

```
List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 1, 3));
Collections.sort(numbers, (a, b) -> b - a);
System.out.println(numbers); // [5, 3, 1]
```

3. Using Collections.sort() with Custom Classes

```
class Student implements Comparable<Student> {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public int compareTo(Student other) {
        return this.age - other.age; // sort by age ascending
    }

    @Override
    public String toString() {
        return name + "(" + age + ")";
```

```
    }
}

List<Student> students = new ArrayList<>();
students.add(new Student("Alice", 22));
students.add(new Student("Bob", 20));
students.add(new Student("Charlie", 25));

Collections.sort(students);
System.out.println(students); // [Bob(20), Alice(22), Charlie(25)]
```

Store primitive types

- Collections in Java only store objects — not primitive types.

Wrapper class

- A wrapper class “wraps” a primitive data type into an object, so we can use it in places where an object is required.

Primitive	Wrapper Class	Boxing (Primitive → Object)	Unboxing (Object → Primitive)	Parsing (String → Primitive)	Auto-boxing / Auto-unboxing
int	Integer	Integer.valueOf(int i)	intValue()	Integer.parseInt(String s)	Integer i = 20; int z = i;
long	Long	Long.valueOf(long l)	longValue()	Long.parseLong(String s)	Long l = 100L; long m = l;
double	Double	Double.valueOf(double d)	doubleValue()	Double.parseDouble(String s)	Double d = 2.5; double e = d;
char	Character	Character.valueOf(char c)	charValue()	❌ No parseChar() (use charAt(0))	Character c = 'A'; char ch = c;
boolean	Boolean	Boolean.valueOf(boolean b)	booleanValue()	Boolean.parseBoolean(String s)	Boolean b = true; boolean flag = b;